

Unit 09: Testing Strategies

CONTENTS

Objectives

Introduction

9.1 Black Box Testing

9.2 Black Box Testing Techniques

9.3 White Box Testing

9.4 White Box Testing Techniques

9.5 Sanity Testing

9.6 Smoke Testing

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objectives

After studying this unit, you will be able to:

- describe black-box testing
- white-box testing
- sanity testing and smoke testing

Introduction

Testing is the process of running software with real or simulated inputs to show that it meets a product's criteria and, if it doesn't, to pinpoint the exact disparities between expected and actual outcomes. Software testing is the process of comparing a piece of software to user and system standards.

Any action aimed at analysing an attribute or capability of a programme or system and deciding if it reaches its necessary results is known as software testing. Software testing, despite being critical to software quality and widely used by programmers and testers, is still considered an art due to a lack of understanding of software concepts. The difficulty in software testing derives from the software's complexity: we can't test a programme of moderate complexity entirely. Debugging is only one aspect of testing. Testing can be done for quality assurance, verification and validation, or to estimate reliability. Testing can be used as a generic metric as well. Correctness testing and reliability testing are two major areas of testing. Software testing is a trade-off between budget, time and quality.

White box and black box testing are terms used to describe the point of view a test engineer takes when designing test cases. Black box is an external view of the test object and white box, an internal view

9.1 Black Box Testing

Black-box testing, also known as behavioral testing, focuses on the software's functional requirements. In other words, black-box testing allows a software engineer to create sets of input conditions that fully exercise all of a program's functional requirements. Black-box testing is not a

replacement for white-box testing. Rather, it's a supplement to white-box approaches, and it's more likely to find a different set of problems.

Black box testing takes an external perspective of the test object to derive test cases. These tests can be functional or non-functional, though usually functional. The test designer selects valid and invalid input and determines the correct output. There is no knowledge of the test object's internal structure.

This test design method can be used at all stages of development, including unit, integration, system, and acceptance. The higher the level, and hence the larger and more complex the box, the more we must rely on black box testing to simplify. While this method can reveal parts of the specification that haven't been implemented, you can't be sure that all existing paths have been tested. It enables a software developer to create a set of inputs that can fully encompass all of a program's functional needs.

Black-box testing attempts to find errors in the following categories: Notes

1. Incorrect or missing functions
2. Interface errors
3. Errors in data structures or external data base access
4. Behavior or performance errors
5. Initialization and termination errors.

By applying black-box techniques, we derive a set of test cases that satisfy the following criteria:

1. Test cases that reduce, by a count that is greater than one, the number of additional test cases that must be designed to achieve reasonable testing.
2. Test cases that tell us something about the presence or absence of classes of errors, rather than an error associated only with the specific test at hand.

Types of Black Box Testing

- Functional testing
- Non-functional testing
- Regression testing

Functional testing: This is a type of black box testing which is related to the functional requirements of a system. Functional testing is concerned only with the functional requirements of a system and covers how well the system executes its functions.

Non-functional testing: This black box testing type is not related to testing of specific functionality, non-functional testing is concerned with the non-functional requirements and is designed specifically to evaluate the readiness of a system according to the various criteria which are not covered by functional testing.

Regression testing: Regression Testing is performed after code fixes, upgrades or any other system maintenance to check the new changes has not affected any existing functionality.

9.2 Black Box Testing Techniques

- Equivalence Class Partitioning
- Boundary Value Analysis
- Cause effect Graphing
- Compatibility testing
- Syntax Driven Testing
- State Transition Testing

Equivalence Class Partitioning

The domain of input values to a programme is partitioned into a set of equivalence classes in this approach. This partitioning is done in such a way that the program's behaviour is consistent across all input data of the same equivalence class.

The idea is to partition the input domain of the system into a number of equivalence classes such that each member of class works in a similar way, i.e., if a test case in one class results in some error, other members of class would also result into same error.

The domain of input values to a programme is partitioned into a set of equivalence classes in this approach. This partitioning is done so that the program's behaviour is consistent across all input data of the same equivalence class. The primary notion behind the equivalence classes is that testing code with any one value from an equivalence class is equivalent to testing software with any other value from that equivalence class. Examining the input and output data can be used to create equivalence classes for software. Some general guidelines for creating equivalence classes are listed below.

1. If the input data values to a system can be specified by a range of values, then one valid and two invalid equivalence classes should be defined.
2. If the input data assumes values from a set of discrete members of some domain, then one equivalence class for valid input values and another equivalence class for invalid input values should be defined.

The technique involves two steps

Identification of equivalence class – Partition any input domain into minimum two sets: valid values and invalid values. For example, if the valid range is 0 to 50 then select one valid input like 45 and one invalid like 69.

Generating test cases - To each valid and invalid class of input assign unique identification number. Write test case covering all valid and invalid test case considering that no two invalid inputs mask each other.



For a software that computes the square root of an input integer which can assume values in the range of 0 to 5000, there are three equivalence classes: The set of negative integers, the set of integers in the range of 0 and 5000, and the integers larger than 5000. Therefore, the test cases must include representatives for each of the three equivalence classes and a possible test set can be: {-5, 500, 6000}.



Design the black-box test suite for the following program. The program computes the intersection point of two straight lines and displays the result. It reads two integer pairs (m1, c1) and (m2, c2) defining the two straight lines of the form $y = mx + c$.

The equivalence classes are the following:

- Parallel lines ($m_1 = m_2, c_1 \neq c_2$)
- Intersecting lines ($m_1 \neq m_2$)
- Coincident lines ($m_1 = m_2, c_1 = c_2$)

Now, selecting one representative value from each equivalence class, the test suit (2, 2) (2, 5), (5, 5) (7, 7), (10, 10) (10, 10) are obtained.

Boundary Value Analysis

Boundary value analysis is a test case design technique that complements equivalence partitioning. Rather than selecting any element of an equivalence class, BVA leads to the selection of test cases at the “edges” of the class. Rather than focusing solely on input conditions, BVA derives test cases from the output domain as well

The values at the boundaries are the focus of boundary value testing. This method assesses whether or not a set of values is acceptable to the system. It's a great way to cut down on the amount of test cases. It's best for systems with inputs that fall within defined parameters.

Boundary value analysis is a software testing technique for determining test cases that address known areas of common problems at the edges of software component input ranges. The borders of input ranges to a software component, in particular, are prone to faults, according to testing experience. A programmer who has to implement the range 1 to 12 at an input, which may be used to represent the months January to December in a date, has a line in his code that checks for this range. This may appear to be:

```
if (month > 0 && month < 13)
```

But a common programming error may check a wrong range e.g. starting the range at 0 by writing:

```
if (month >= 0 && month < 13)
```

For more complex range checks in a program this may be a problem which is not so easily spotted as in the above simple example.

Guidelines for BVA are similar in many respects to those provided for equivalence partitioning:

1. If an input condition specifies a range bounded by values a and b, test cases should be designed with values a and b and just above and just below a and b.
2. If an input condition specifies a number of values, test cases should be developed that exercise the minimum and maximum numbers.
3. Apply guidelines 1 and 2 to output conditions.



Assume that a temperature vs. pressure table is required as output from an engineering analysis program.

Test cases should be designed to create an output report that produces the maximum (and minimum) allowable number of table entries.

4. If internal program data structures have prescribed boundaries (e.g., an array has a defined limit of 100 entries), be certain to design a test case to exercise the data structure at its boundary.

Most software engineers intuitively perform BVA to some degree. By applying these guidelines, boundary testing will be more complete, thereby having a higher likelihood for error detection.



For a function that computes the square root of integer values in the range of 0 and

5000, the test cases must include the following values: {0, -1,5000,5001}.

Cause effect Graphing

One weakness with the equivalence class partitioning and boundary value methods is that they consider each input separately. That is, both concentrate on the conditions and classes of one input. They do not consider combinations of input circumstances that may form interesting situations that should be tested.

Consider all valid combinations of the equivalence classes of input conditions as one approach to exercise combinations of distinct input circumstances. This straightforward approach will generate a disproportionately large number of test cases, many of which will be useless in exposing new errors. For example, if there are n different input conditions, we will have 2^n test cases because any combination of the input conditions is legitimate. Cause-effect graphing is a technique for selecting input conditions in a systematic manner so that the number of test cases does not grow unmanageably huge. The technique starts with identifying causes and effects of the system under testing. A cause is a distinct input condition, and an effect is a distinct output condition. Each condition forms a node in the cause effect graph.

The conditions should be written in such a way that they can be true or untrue. For example, an input condition could be "file is empty," which would be true if the input file was empty and false if it wasn't. Following the identification of causes and effects, we determine the causes that can produce each effect and how the conditions must be coupled to make the effect true for each effect. The Boolean operators "and," "or," and "not," which are represented in the graph by a $\wedge$, $\vee$, and zigzag line, respectively, are used to combine conditions.

Then, for each effect, all combinations of the causes that the effect depends on which will make the effect true, are generated (the causes that the effect does not depend on are essentially "don't care"). By doing this, we identify the combinations of conditions that make different effects true. A test case is then generated for each combination of conditions, which make some effect true.

Let us illustrate this technique with a small example. Suppose that for a bank database there are two commands allowed:

credit acct-number transaction amount

debit acct-number transaction amount

The account gets credited if the command is credit and the acct-number is correct. The account gets debited if the command is debit, the account number is valid, and the transaction amount is valid (less than the balance). A suitable message is generated if the command is invalid, the account number is invalid, or the debit amount is invalid. From these conditions, we may deduce the following causes and consequences:

Cause:

- c1. Command is credit
- c2. Command is debit
- c3. Account number is valid
- c4. Transaction amt. is valid

Effects:

- e1. Print "invalid command"
- e2. Print "invalid account-number"
- e3. Print "Debit amount not valid"
- e4. Debit account
- e5. Credit account

This technique establishes relationship between logical inputs called causes with corresponding actions called effect. The causes and effects are represented using Boolean graphs.

Identify inputs (causes) and outputs (effect).

Develop cause effect graph.

Transform the graph into decision table.

Convert decision table rules to test cases.

Compatibility testing

The test case result not only depend on product but also infrastructure for delivering functionality. When the infrastructure parameters are changed it is still expected to work properly

Syntax Driven Testing

This type of testing is applied to systems that can be syntactically represented by some language. For example- compilers, language that can be represented by context free grammar. In this, the test cases are generated so that each grammar rule is used at least once.

State Transition Testing

This testing technique uses the inputs, outputs, and the state of the system during the testing phase. It checks the software against the sequence of transitions or events among the test data.

Steps of black box testing

- The first step to black-box testing is to understand the requirement specifications of the application under test.
- The next step is to evaluate the set of valid inputs and test scenarios to test the software
- Prepare the test cases to cover a maximum range of inputs.
- The test cases are run in the system to generate output, which is validated with the expected outcome to mark pass or fail.
- The failed steps are marked and sent to the development team to fix them.
- Retest the system using various testing techniques to verify its recurring nature or to pass it.

Black Box Testing and SDLC

Requirement - This is the initial stage of SDLC and in this stage, a requirement is gathered. Software testers also take part in this stage.

Test Planning and Analysis - Testing Types applicable to the project are determined. A Test Plan is created which determines possible project risks and their mitigation.

Design - In this stage Test cases/scripts are created on the basis of software requirement documents

Test Execution- In this stage Test Cases prepared are executed. Bugs if any are fixed and re-tested.

Advantages of Black Box testing

- The tester doesn't need any technical knowledge to test the system. It is essential to understand the user's perspective.
- Testing is performed after development, and both the activities are independent of each other.
- It works for a more extensive coverage which is usually missed out by testers as they fail to see the bigger picture of the software.

Disadvantages of Black-box Testing

- The test inputs need to be from large sample space.
- It is difficult to identify all possible inputs in limited testing time. So writing test cases is slow and difficult.
- Chances of having unidentified paths during this testing. Test cases can be generated before development and right after specification.

9.3 White Box Testing

White box testing (also known as clear box testing, glass box testing, or structural testing) designs test cases based on the internal structure of the system. To identify all paths across the software, you'll need programming expertise. The tester selects test case inputs and finds the relevant outputs for all paths. Every node in a circuit can be probed and measured in electrical hardware testing, for example, in circuit testing (ICT).

Since the tests are based on the actual implementation, if the implementation changes, the tests probably will need to also. For example ICT needs updates if component values change, and needs modified/new fixture if the circuit changes.

This adds financial resistance to the change process, thus buggy products may stay buggy. Automated optical inspection (AOI) offers similar component level correctness checking without the cost of ICT fixtures, however changes still require test updates.

While white box testing can be used at the unit, integration, and system levels, it is most commonly used at the unit level. While it is typically used to test paths within a unit, it can also be used to test paths between units during integration and subsystems during a system level test. Though this type of test design can reveal a large number of test cases, it may miss sections of the specification that haven't been implemented or criteria that haven't been met. You can be certain, however, that all paths through the test object are followed.

White-box testing or glass-box testing is a test case design method that uses the control structure of the procedural design to obtain test cases. Such kind of testing, software engineer can be deriving test cases which:

1. Guarantee that all independent paths within a module have been exercised at least once,
2. Exercise all logical decisions on their true and false sides
3. Execute all loops at their boundaries and within their operational bounds
4. Exercise internal data structures to ensure their validity.

Steps for white box testing

Input: Requirements, Functional specifications, design documents, source code.

Processing: Performing risk analysis for guiding through the entire process.

Test planning: Designing test cases so as to cover entire code. Execute test plan until error-free software is reached

Output: Preparing final report of the entire testing process.

9.4 White Box Testing Techniques

- Statement Coverage
- Branch Coverage
- Path Coverage
- Condition Coverage
- Loop Testing

Statement Coverage

The statement coverage strategy aims to design test cases so that every statement in a program is executed at least once. The principal idea governing the statement coverage strategy is that unless a statement is executed, it is very hard to determine if an error exists in that statement. Unless a statement is executed, it is very difficult to observe whether it causes failure due to some illegal memory access, wrong result computation, etc. However, executing some statement once and observing that it behaves properly for that input value is no guarantee that it will behave correctly for all input values. In the following, designing of test cases using the statement coverage strategy have been shown.

Example: Consider the Euclid's GCD computation algorithm:

```
int compute_gcd (x, y)
int x, y; {1 while (x!= y)
{2 if (x>y) then, 3 x= x - y; 4 else y= y - x; 5}6 return x;}
```

By choosing the test set {(x=3, y=3), (x=4, y=3), (x=3, y=4)}, we can exercise the program such that all statements are executed at least once.

Branch Coverage

In this technique, test cases are designed so that each branch from all decision points are traversed at least once. In a flowchart, all edges must be traversed at least once.

Path Testing

In this technique, control flow graphs are made from code or flowchart and then Cyclomatic complexity is calculated which defines the number of independent paths so that the minimal number of test cases can be designed for each independent path.

This technique corresponds to testing all possible paths which means that each statement and branch is covered.

Path testing steps

- Make the corresponding control flow graph
- Calculate the cyclomatic complexity
- Find the independent paths
- Design test cases corresponding to each independent path

Condition Coverage

In this technique, all individual conditions must be covered



READ A, B

IF (A == 0 || B == 0)

PRINT '0'

In this example, there are 2 conditions: A == 0 and B == 0. Now, test these conditions get TRUE and FALSE as their values.

Loop Testing

Loops are widely used and these are fundamental to many algorithms hence, their testing is very important. Errors often occur at the beginnings and ends of loops. Loops are very important constructs for generally all the algorithms. Loop testing is a white box testing technique. It focuses exclusively on the validity of loop constructs.

Simple loops

Nested loops

Concatenated loops

Simple loop: The following set of tests should be applied to simple loop where n is the maximum number of allowable passes thru the loop:

- Skip the loop entirely.

- Only one pass thru the loop.
- Two passes thru the loop.
- M passes thru the loop where $m < n$.
- N-1, n, n+1 passes thru the loop.

Nested loop: Beizer approach to the nested loop is:

- Start at the innermost loop. Set all other loops to minimum value.
- Conduct the simple loop test for the innermost loop while holding the outer loops at their minimum iteration parameter value.
- Work outward, conducting tests for next loop, but keeping all other outer loops at minimum values and other nested loops to typical values.
- Continue until all loops have been tested.

Concatenated loops: These can be tested using the approach of simple loops if each loop is independent of other. However, if the loop counter of loop 1 is used as the initial value for loop 2 then approach of nested loop is to be used.

Unstructured loop: This class of loops should be redesigned to reflect the use of the structured programming constructs.

Types of White Box Testing

Unit testing – tests written as part of the application code, which test that each component is working as expected.

Mutation testing – a type of unit testing that checks the robustness and consistency of the code by defining tests, making small, random changes to the code and seeing if the tests still pass.

Integration testing – tests specifically designed to check integration points between internal components in a software system, or integrations with external systems.

Static code analysis – automatically identifying vulnerabilities or coding errors in static code, using predefined patterns or machine learning analysis.

White Box Penetration Testing: In this testing, the tester/developer has full information of the application's source code, detailed network information, IP addresses involved and all server information the application runs on. The aim is to attack the code from several angles to expose security threats.

White Box Testing Tools

- NUnit
- PyUnit
- Parasoft Jtest
- EcEmma
- HTMLUnit
- CppUnit

Advantages of White box testing

- White box testing optimizes code so hidden errors can be identified.
- Test cases of white box testing can be easily automated.
- This testing is more thorough than other testing approaches as it covers all code paths.
- It can be started in the SDLC phase even without GUI.

Disadvantages of White box testing

- White box testing is too much time consuming when it comes to large-scale programming applications.
- White box testing is much expensive and complex.
- It can lead to production error because it is not detailed by the developers.

9.5 Sanity Testing

Sanity testing is a type of test that is performed to touch on each implementation and its impact but not in-depth or comprehensively; it may encompass functional, UI, version, and other aspects. After obtaining a software build with minor modifications in code or functionality, sanity testing is performed to ensure that the bugs have been fixed and that no new issues have been introduced as a result of the changes.

The objective is "not" to verify thoroughly the new functionality but to determine that the developer has applied some rationality (sanity) while producing the software. If sanity test fails, the build is rejected to save the time and costs involved in a more rigorous testing.

Sanity testing was performed when we are receiving software build (with minor code changes) from the development team. It is a checkpoint to assess if testing for the build can proceed or not. Sanity testing also ensures that the modification in the code or functions does not affect the associated modules. Consequently, it can be applied only on connected modules that can be impacted.

Features of Sanity Testing

Subset of Regression Testing: Sanity testing is a subset of regression testing and focuses on the smaller section of the application.

Unscripted: Most of the times sanity testing is not scripted.

Not documented: Usually sanity testing is undocumented.

Narrow and deep: Sanity testing is narrow and deep approach of testing where limited functionalities are covered deeply.

Performed by testers: Sanity testing is normally performed by testers

Sanity Testing Process

The main purpose of performing sanity testing is to check the incorrect outcomes or defects which are not existing in component procedures. And also, ensure that the newly added features may not affect the functionalities of current features.

Therefore, we need to follow the below steps to implement the sanity testing process gradually:

- Identification
- Evaluation
- Testing

Identification

The first step in the sanity testing process is Identification, where we detect the newly added components and features as well as the modification presented in the code while fixing the bug.

Evaluation

After completing the identification step, we will analyze newly implemented components, attributes and modify them to check their intended and appropriate working as per the given requirements.

Testing

Once the identification and evaluation step are successfully processed, we will move to the next step, which is testing. In this step, we inspect and assess all the linked parameters, components, and essentials of the above analyzed attributed and modified them to make sure that they are working fine.

Advantages of Sanity Testing

- Sanity testing helps in quickly identify defects in the core functionality
- It is less expensive as compared to other types of software testing
- It can be carried out in lesser time as no documentation is required for sanity testing.
- If the defects are found during sanity testing, project is rejected that is helpful in saving time for execution of regression tests.

Disadvantages of Sanity testing

- All the test cases are not covered under sanity testing.
- It is emphasized only on the statement and functions of the application.

- We do not have future references since the sanity testing is unscripted.



Suppose we have an e-commerce application, which contains several modules, but here, we mainly concentrate only a few modules such as the login page, the home page, the new user creation page, the user profile page, etc.

While a new user tries to login into the application, he/she is not able to log in, as there is a bug in the login page.

Because the password field in the login module accepts less than four alpha-numeric characters and based on the specification, the password field should not be accepted below 7-8 characters.

Thus, it is considered as bug, which is reported by the testing team to the development team to fix it.

Once the development team fixes the specified bug and reports back to the testing team, the testing team tests the same feature to verify that the modification that happens in the code is working fine or not.

And the testing team also verifies that the particular modification does not impact other related functionalities.

To modify the password on the user profile page there is a process.

As part of the sanity testing process, we must authenticate the login page and the profile page to confirm that the changes are working fine at both the places.

9.6 Smoke Testing

Smoke Testing is a type of software testing which is usually performed on initial software builds to make sure that the critical functionalities of the program are working fine. It is executed before any detailed functional or regression tests are executed.

Smoke Testing is a software testing process that determines whether the deployed software build is stable or not. Smoke testing is a confirmation for QA team to proceed with further software testing. It consists of a minimal set of tests run on each build to test software functionalities. Smoke testing is also known as "Build Verification Testing" or "Confidence Testing."

Characteristics of Smoke Testing

- Smoke testing is documented.
- Smoke testing may be stable as well as unstable.
- Smoke testing is scripted.
- Smoke testing is type of regression testing.

Steps for smoke testing

- Identify smoke test cases
- Create smoke tests
- Run smoke test
- Analyze smoke tests

Types of Smoke Testing

Manual method: In general, smoke testing is done manually. It approaches varies from one organization to other. Smoke testing is carried to ensure the navigation of critical paths is as expected and doesn't hamper the functionality. Once the build is released to QA, high priority functionality test cases are to be taken and are tested to find the critical defects in the system. If the test passes, we continue the functional testing. If the test fails, the build is rejected and sent back to the development team for correction. QA again starts smoke testing with a new build version. Smoke testing is performed on new build and will get integrated with old builds to maintain the correctness of the system. Before performing smoke testing, QA team should check for correct build versions.

Automation method: Automation Testing is used for Regression Testing. However, we can also use a set of automated test cases to run against Smoke Test. With the help of automation tests, developers can check build immediately, whenever there is a new build ready for deployment.

Instead of having repeated test manually whenever the new software build is deployed, recorded smoke test cases are executed against the build. It verifies whether the major functionalities still

operates properly. If the test fails, then they can correct the build and redeploy the build immediately. By this, we can save time and ensure a quality build to the QA environment.

Using an automated tool, test engineer records all manual steps that are performed in the software build.

Advantages of Smoke Testing

It saves test effort and time.

It makes easy to detect critical errors and helps in correction of errors.

It runs quickly.

It minimizes integration risks.

Summary

- Tests must be conducted to find the highest possible number of errors, must be done systematically and in a disciplined way.
- Testing software can be considered as the only destructive (psychologically) step in the entire life cycle of software production.
- A program developed should be testable i.e. it should be possible to test the program.
- An oracle is a mechanism for determining whether the program has passed or failed a test.
- One of the key problems with oracles is that they can only address a small subset of the inputs and outputs actually associated with any test.
- A test case is a set of conditions or variables under which a tester will determine whether a system under test satisfies requirements or works correctly.
- There are two fundamental properties for a testing criterion: reliability and validity.
- Black-box testing, also called behavioral testing, focuses on the functional requirements of the software.
- Equivalence partitioning is a black-box testing method that divides the input domain of a program into classes of data from which test cases can be derived.
- White box-testing referred to as glass box test can be defined as a test case design method which employs control structure of procedural design in order to derive test cases.

Keywords

Basis Path Testing: It allows the design and definition of a basis set of execution paths.

Black-box Testing: It refers to tests that are conducted at the software interfaces; it mainly focuses on the functional requirements of the software.

Boundary Value Analysis: Boundary value analysis is a test case design technique that complements equivalence partitioning.

Data Flow Testing: The data flow testing method selects test paths of a program according to the locations of definitions and uses of variables in the program.

Equivalence Class: An equivalence class represents a set of valid or invalid states for input conditions.

Equivalence Partitioning: Equivalence partitioning is a black-box testing method that divides the input domain of a program into classes of data from which test cases can be derived.

Failure: Failure is the inability of a system or component to perform a required function according to its specifications.

Fault: Fault is a condition that causes a system to fail in performing its required function.

Loop Testing: Loop testing is a white-box testing technique that focuses exclusively on the validity of loop constructs.

Software Testing: Software testing is an investigation conducted to provide stakeholders with information about the quality of the product or service under test.

Statement Coverage: Statement coverage is a measure of the percentage of statements that have been executed by test cases.

White-box Testing: It works on the principle of closely monitoring the procedural details of the software.

Self Assessment

1. Black box testing is ____
 - A. Focuses on the functional requirements of the software
 - B. No knowledge required for internal code structure, implementation details and internal paths
 - C. It is used to test the system against external factors responsible for software failures
 - D. All of above

2. What are the types of Black Box Testing?
 - A. Functional testing
 - B. Non-functional testing
 - C. Regression testing
 - D. All of above

3. Boundary Value Analysis is part of _
 - A. White box testing
 - B. Black box testing
 - C. Unit testing
 - D. None of above

4. Which is not part of Black Box Testing Techniques?
 - A. Cause effect Graphing
 - B. Compatibility testing
 - C. Load testing
 - D. Syntax Driven Testing

5. Generating test cases and Identification of equivalence class are part of__
 - A. Cause effect Graphing
 - B. Equivalence Class Partitioning
 - C. Syntax Driven Testing
 - D. None of above

6. Cause effect graphing is__
 - A. Identify inputs (causes) and outputs (effect).
 - B. Develop cause effect graph.
 - C. Transform the graph into decision table
 - D. All of above

7. yntax Driven Testing and State Transition Testing is part of ____
 - A. Integration testing
 - B. Unit testing
 - C. White box testing
 - D. Black box testing

8. What are different Loop Testing__?
 - A. Simple
 - B. Nested
 - C. Concatenated
 - D. All of above

9. NUnit and PyUnit is part of ____
- A. Integration testing
 - B. White box testing
 - C. Unit testing
 - D. Black box testing
10. What are the types of Smoke testing?
- A. Manual method
 - B. Automation method
 - C. Hybrid method
 - D. All of above

Answer for Self Assessment

- | | | | | |
|------|------|------|------|-------|
| 1. D | 2. D | 3. B | 4. C | 5. B |
| 6. D | 7. D | 8. D | 9. B | 10. D |

Review Questions

1. What are the fundamentals of software testing?
2. Explain the benefits and objectives of testing.
3. Enumerate the various principles of testing.
4. Discuss about white box testing techniques.
5. How smoke testing is different from sanity testing.
6. What are advantages of white box testing?
7. Briefly explain the Black-box testing. Discuss about equivalence partitioning



Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering – A Practitioner’s Approach, 5th Edition, Tata
- McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education



Online Link

- www.guru99.com/software-testing.html
- www.ece.cmu.edu/~koopman/des_s99/sw_testing/
- www.softwarequalitymethods.com/Papers/OracleTax.pdf
- se.fsksm.uttm.my/.../IST-AutomatedFrameworkSoftTestOracle-FPage.pdf
- <https://www.geeksforgeeks.org/smoke-testing-software-testing/>
- <https://www.javatpoint.com>